Zero-Knowledge Proofs for Floating-Point and Audio Provenance

Tinh Nguyen

Master's Project Presentation
April 17, 2026

Collaborators: Garrett Greiner, Caleb Geren

Committee:
Prof. Pratik Soni
Prof. Ganesh Gopalakrishnan
Prof. Aditya Bhaskara

Outline

Problem

Background

Cryptographic Tools

Baselines

Experiments & Results

End-to-end Demo

Problem

Verifying authenticity and provenance of media is becoming increasingly important

- Demand for reliably determining the source of media and how it has been altered
- Media goes through many transformations before being served to the end user

How do we verify that edited media still traces back to a trusted source?

Current Approach

C2PA (Coalition for Content Provenance and Authenticity)

- Standard for media signing cryptographically at capture
- Editing software maintains provenance

The Problem?

- Relies on trusting the editing software

Threat Model

Extend C2PA Standard Trust Model

- Original source (camera, microphone, ML server, etc.) is trusted
- Editing software is now treated as untrusted

We want to verify provenance of media after it has been transformed by an untrusted party without knowing the original.

Outline

Problem

Background

Cryptographic Tools

Baselines

Experiments & Results

End-to-end Demo

Why Audio?

Data representation - We have to prove about integer and floating point numbers.

Transformations - We can represent audio data as linear operations.

Time series data - Data grows with time in the audio.

Use cases - Journalism, testimonies, podcasts

How do we represent audio?

Audio is a **sequence of samples**

audio $x = [x_1, x_2, x_3, x_4, \ldots x_n]$

Each sample represents the **amplitude** (how loud it is, direction) of the sound wave at certain point in time

Notes:

The length of the audio in time grows with number of samples (kHz)

- Example: 44.1 kHz audio = 44,100 samples per second

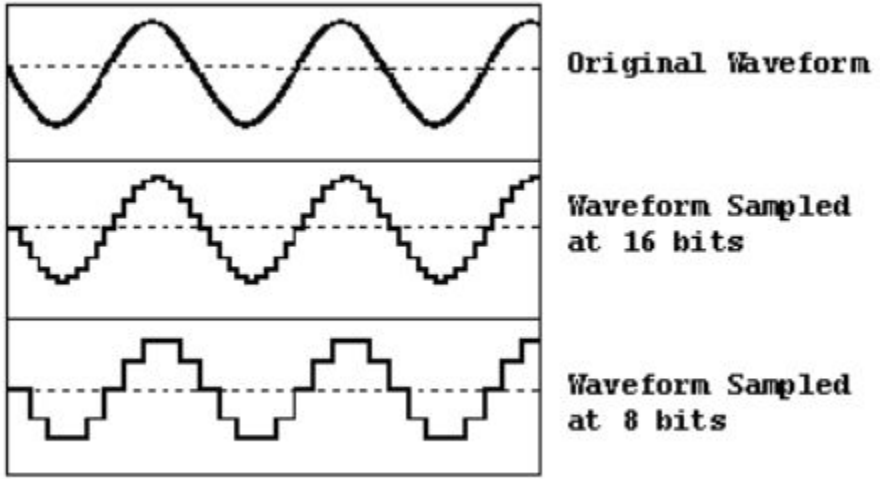
Audio can have different channels (L, R) - parallel sequences

How do we represent audio?

Each audio sample is stored as a numerical value

Different formats determine precision and range. Expressiveness of the sound.

- 8-bit int
- 16-bit int
- 24-bit int
- 32-bit float
- 64-bit float



Outline

Problem

Background

Cryptographic Tools

Baselines

Experiments & Results

End-to-end Demo

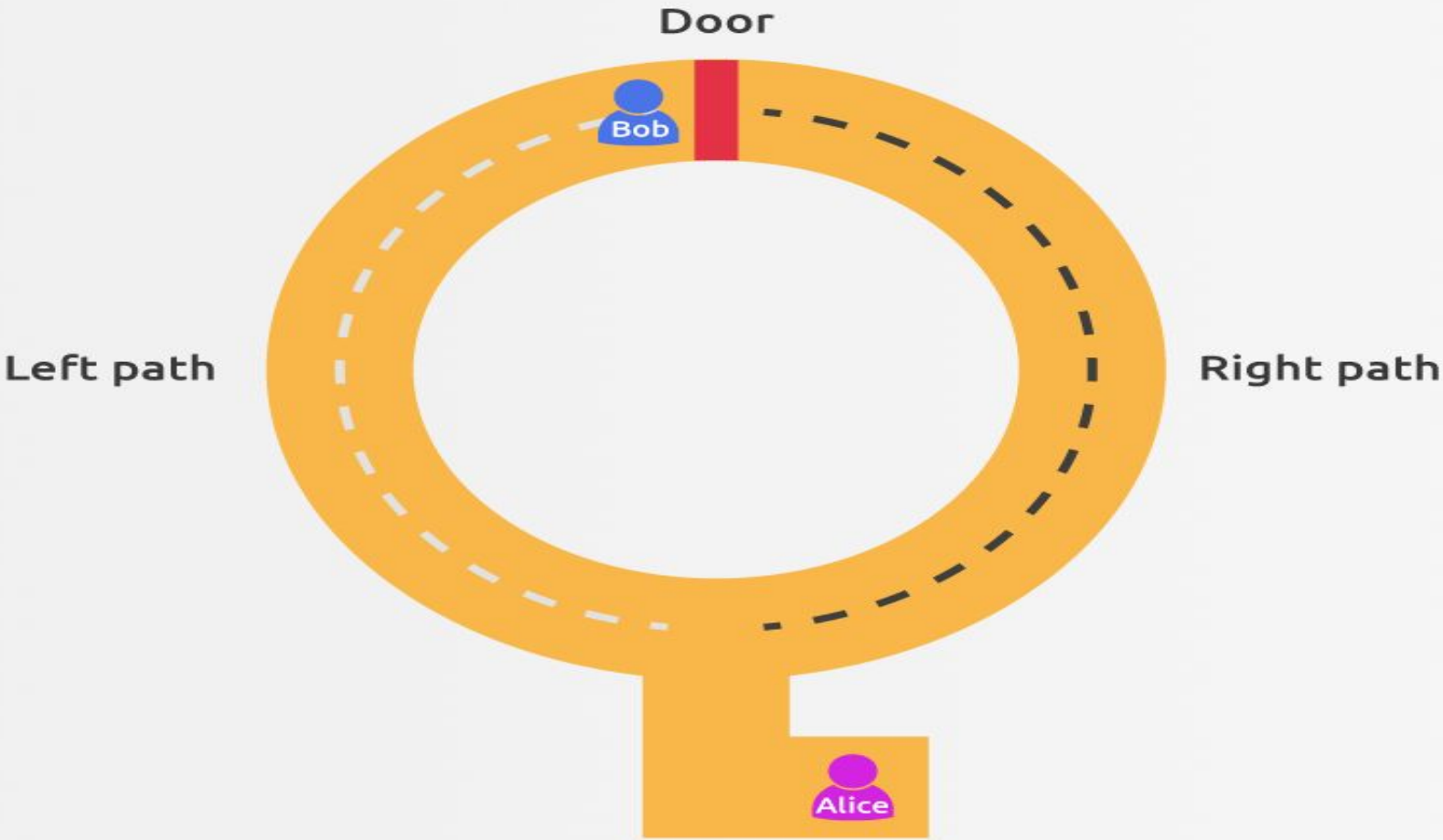
What do we want to prove?

Given an output audio file, we want to prove that it is the result of applying valid transformations to a private audio file.

We need to prove this efficiently and without revealing the original audio

How do we do this?

Zero-Knowledge Proofs (ZKPs)


Door
Bob
Left path
Right path
Alice

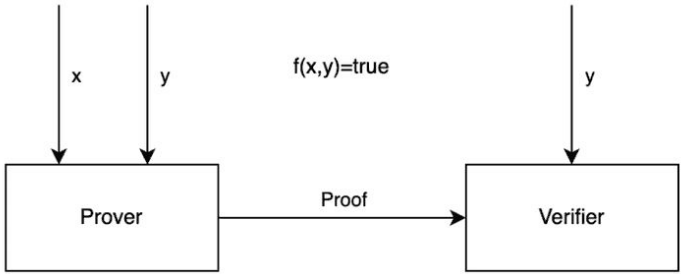
Zero-Knowledge Proofs

Prover:

- Knows secret witness x
- Generates proof π to convince the verifier that it knows x that satisfies f(x,y)

Verifier:

- Has public input y
- Checks proof π with y
- Verifier outputs a decision:
  - Accept “I am convinced”
  - Reject “I am unconvinced”



zk-SNARKS

Completeness - An honest prover can always convince the verifier of a true statement.

Soundness - A dishonest prover cannot convince the verifier of an invalid statement, except with negligible probability.

Zero Knowledge - The proof reveals nothing about the witness, beyond its correctness.

Succinct - Proof size and verification time sublinear in the witness.

How do we construct these zk-SNARKs?

All computations in zk-SNARKS must be represented in arithmetic constraints

R1CS - reduce every computation to constraints of the form $(a \cdot b = c)$

More constraints → slower prover, higher memory

Tying it together

zk-SNARK Construction

Witness: x (original audio)

Public input: y (output audio), T (transformation)

Statement: $y = T(x)$

Outline

Problem

Background

Cryptographic Tools

Baselines

Experiments & Results

End-to-end Demo

Integer Audio

Baseline 1: HyperVerITAS (Greiner et al., 2026) Extension

HyperVerITAS was designed for efficient image provenance

- We extend it from images → audio (2D → 1D)

Adapt the same proof system:

- Pre-image hash proof - input matches signed digest
- Range check - check that samples are in valid range (8-bit int → 8, 16, 24-bit int)
- Transformation correctness - $y = T(x)$

Baseline 1: HyperVerITAS (Greiner et al., 2026) Extension

Pros:

- Fast prove and verify time, scales with large number of samples
- Supports 8, 16, and 24-bit int audio

Cons:

- Integer operations only, cannot handle float audio
- Transformations limited to integer arithmetic

Floating Point Audio

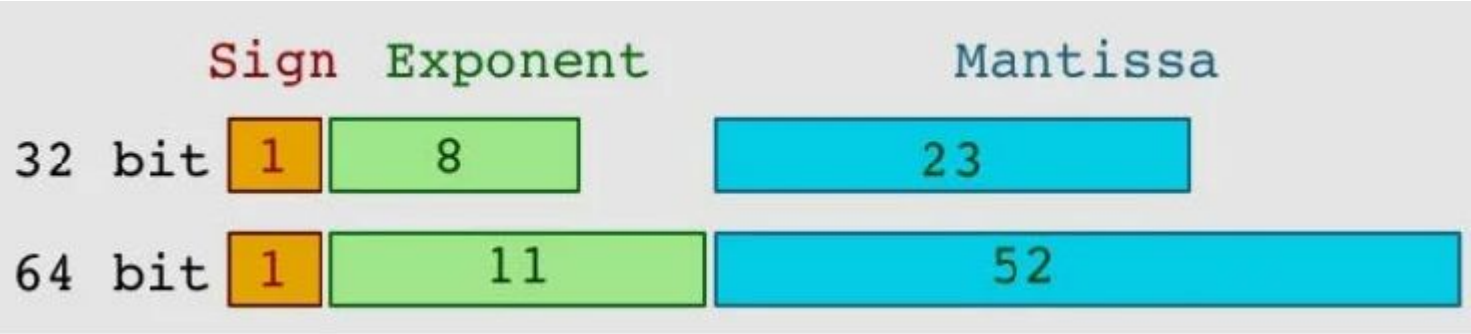
Why are floating point numbers challenging?

In general existing efficient cryptographic tools work with integers, not floating point numbers.

Why don’t we just convert float to integer or fixed point?

- Loss in accuracy and representation provided by floating point
- Converting float to fixed point results in bit width blowup
- Proofs over integers or fixed point don’t bind to floating point computations

Floating Point Number Representation



Baseline 2: ZK Location Privacy (Ernstberger et al., 2024)

Main Idea:

- Full IEEE-754 compliant floating-point ZKP system
- Engineered for practical costs using hints + lookup arguments
    - prover computes decomposition outside the circuit and circuit checks consistency

- Provides sin, cos, sqrt for complex transformations

Baseline 2: ZK Location Privacy (Ernstberger et al., 2024)

Pros:

- IEEE-754 compliant with practical constraint costs
- Provides trig approximation - used for complex transformations

Cons:

- Pay for constraints to achieve IEEE-754 compliance per sample

Can we trade some accuracy for lower cost?

Baseline 3: Succinct Zero Knowledge for Floating Point Computations (Garg et al., 2022)

Main Idea:

- Instead of proving exact IEEE-754 compliance, prove the output is within a relative error bound δ of the true computation

$$|c - g(a, b)| \leq \delta |g(a, b)|$$

- Matches how floating-point correctness is reasoned about in practice (numerical analysis, ML, DSP)

What's the problem?

Baseline 3: Succinct Zero Knowledge for Floating Point Computations (Garg et al., 2022)

No implementation!

Our Implementation: zk-float-compiler

Compiles any R1CS-based proof system into one that supports floating-point operations

How?

- Replace bit decomposition with range proofs
- Prove the error is bounded, not the exact rounding

Faster proving time and lower constraints if multiplication dominates the circuit

Baseline 3: Succinct Zero Knowledge for Floating Point Computations (Garg et al. 2022)

Pros:

- Lower constraint cost than exact IEEE-754 compliance
- Only introduces log(w) overhead for the underlying prover

Cons:

- Only allows addition and multiplication floating point operations.
- Weaker guarantees than exact compliance

Outline

Problem

Background

Cryptographic Tools

Baselines

Experiments & Results

End-to-end Demo

Experiments & Audio Transformations

Volume - Increase or decrease volume by matrix multiplication

Combine - Combine two audio channels or two audio samples by averaging the samples

Trim - Cut out or silence audio

Beep - Add a beep to the audio, uses sin function and division

Tremolo - Add a repeated wavering effect, uses sin, addition, multiplication, and division

Experiments & Audio Transformations

Setting

Benchmarks for addition, multiplication circuits to directly compare.

Additionally benchmarked beep which includes sin and division circuits.

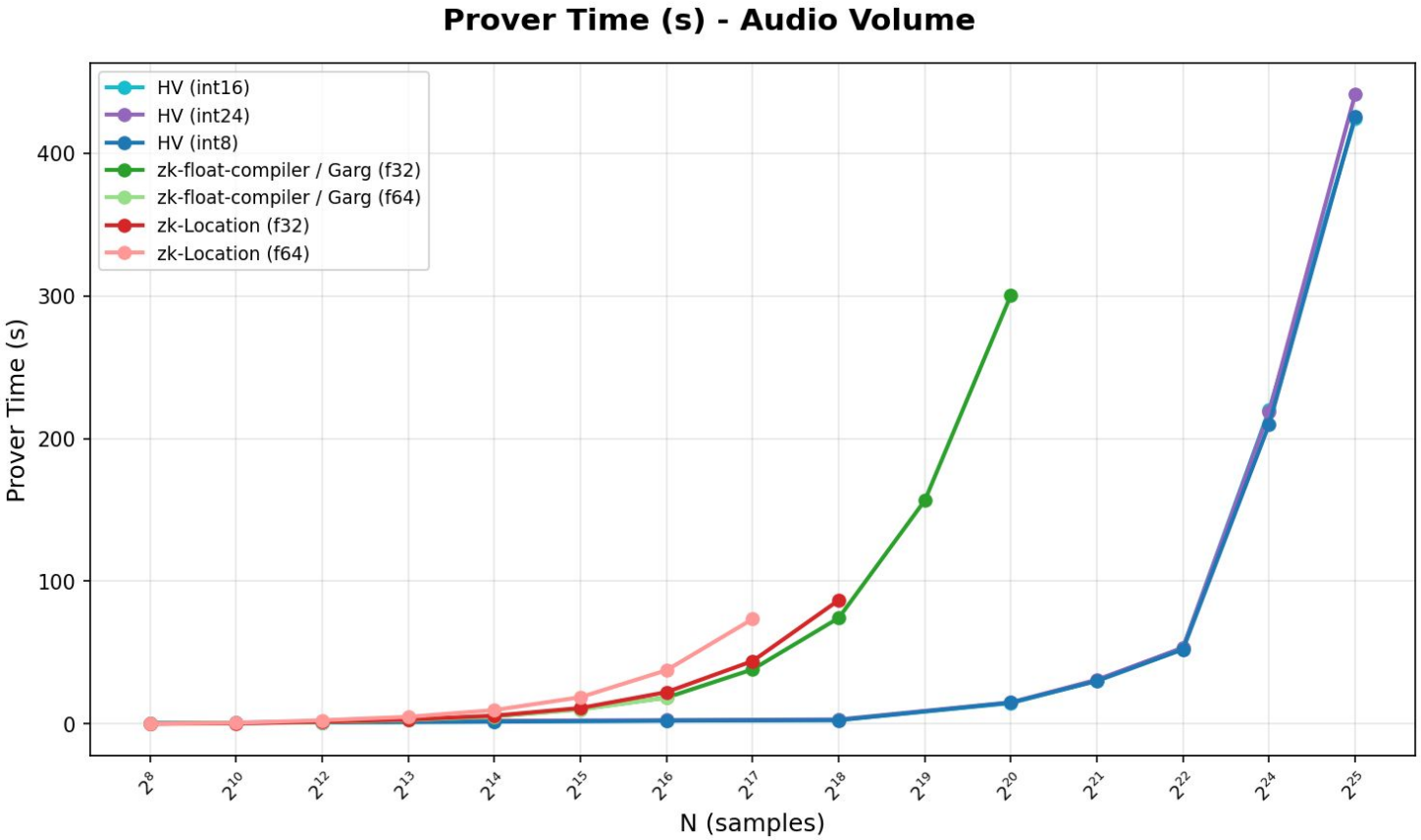
AWS r5d.4xlarge 128GB Memory 16vCPUs

Volume: y[i] = a * x[i]

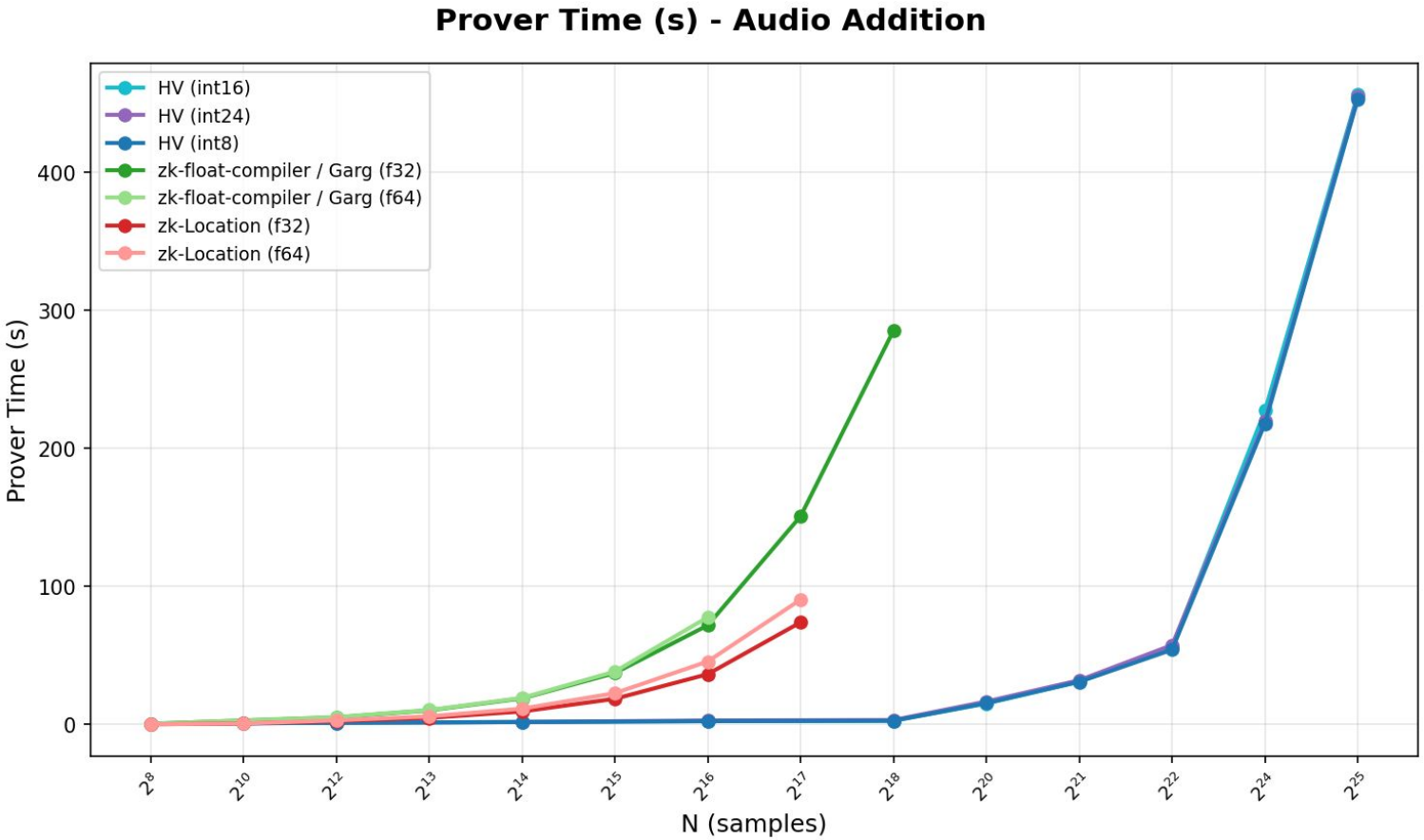
Addition: y[i] = a[i] + b[i]

Beep: y[i] = x[i] + sin(2pif*i / sr)

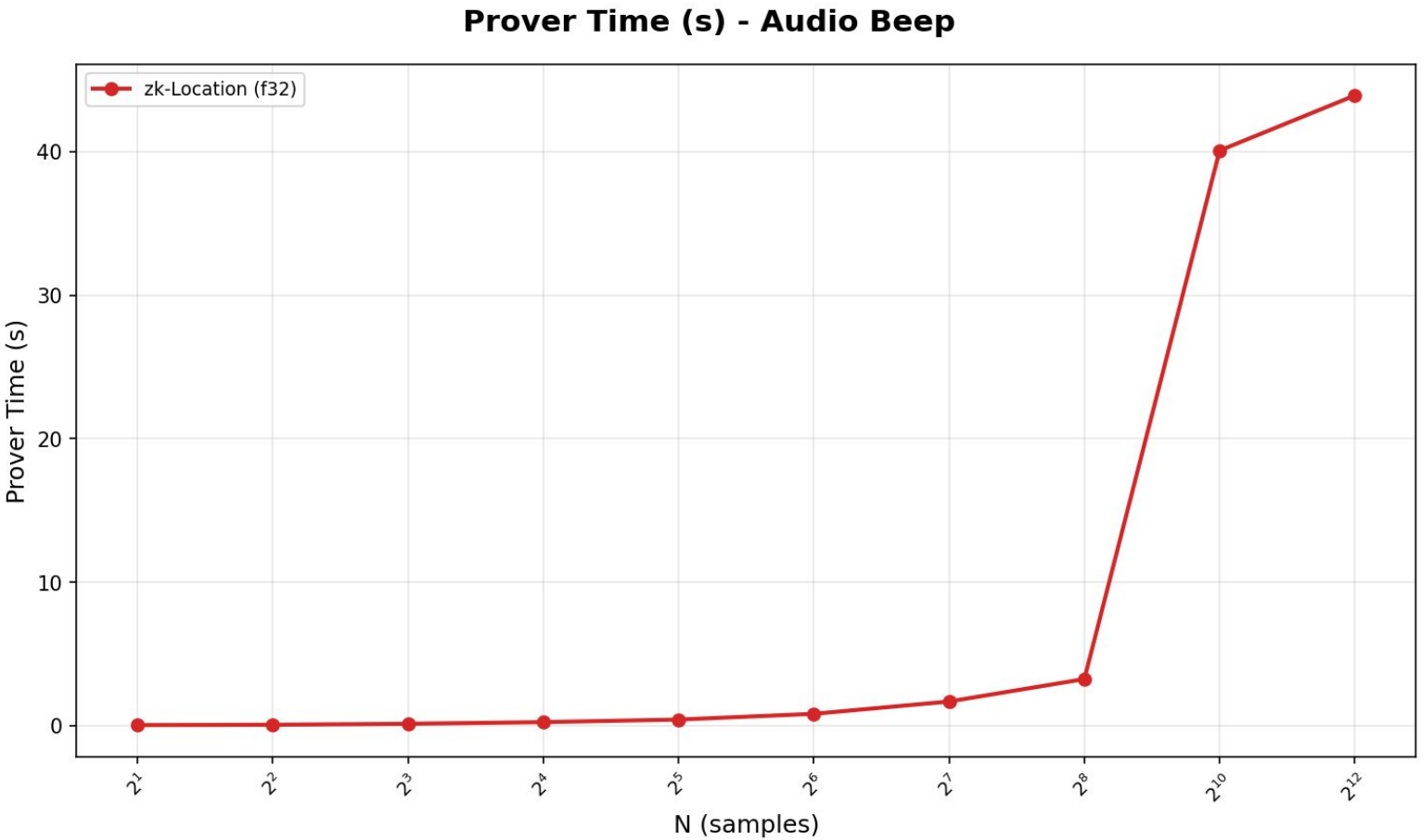
Results


Prover Time (s) - Audio Volume
HV (int16)
HV (int24)
HV (int8)
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Prover Time (s)
0
100
200
300
400
N (samples)
2^{8}
2^{10}
2^{12}
2^{13}
2^{14}
2^{15}
2^{16}
2^{17}
2^{18}
2^{19}
2^{20}
2^{21}
2^{22}
2^{24}
2^{25}

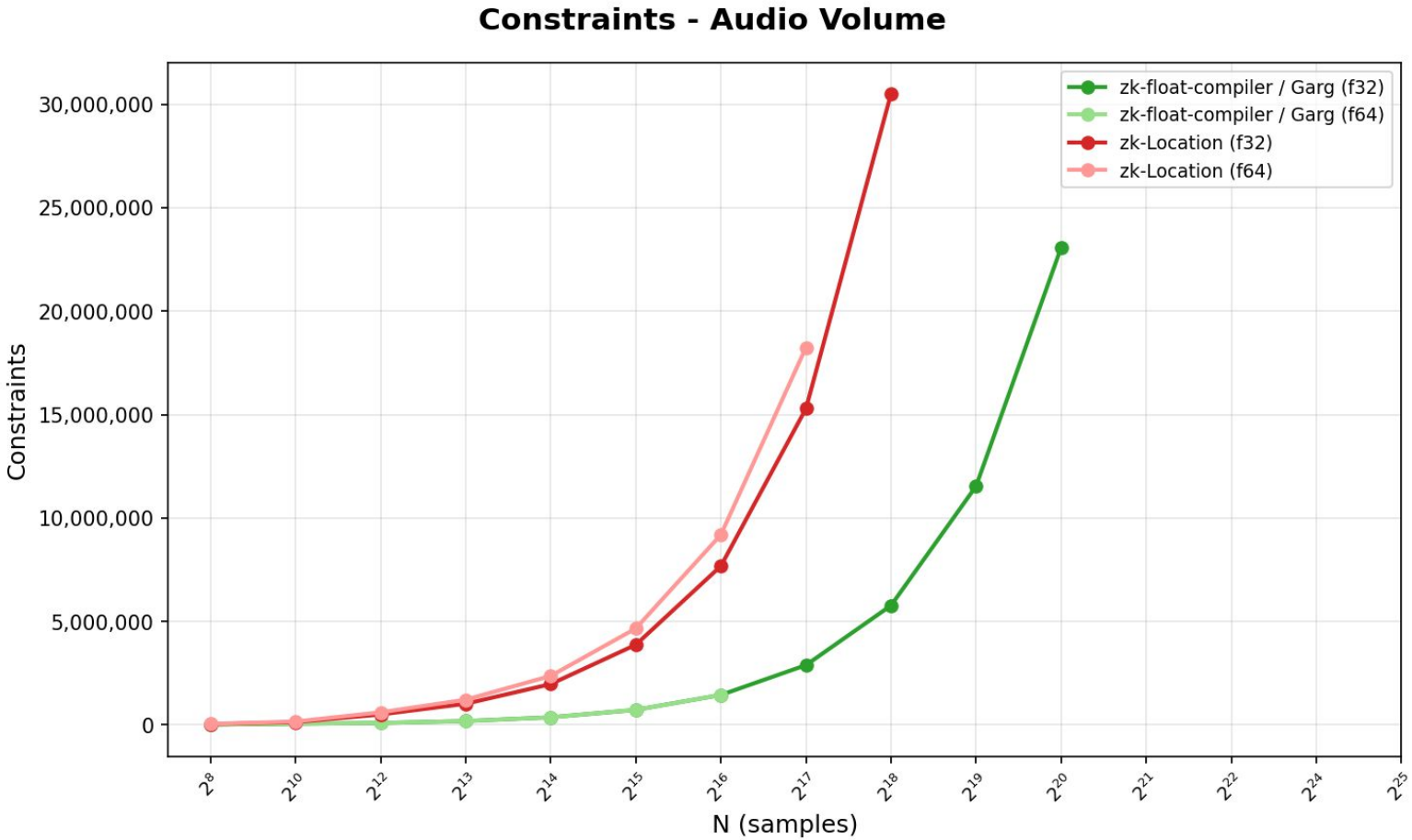
Results


Prover Time (s) - Audio Addition
HV (int16)
HV (int24)
HV (int8)
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Prover Time (s)
0
100
200
300
400
N (samples)
2^{8}
2^{10}
2^{12}
2^{13}
2^{14}
2^{15}
2^{16}
2^{17}
2^{18}
2^{20}
2^{21}
2^{22}
2^{24}
2^{25}

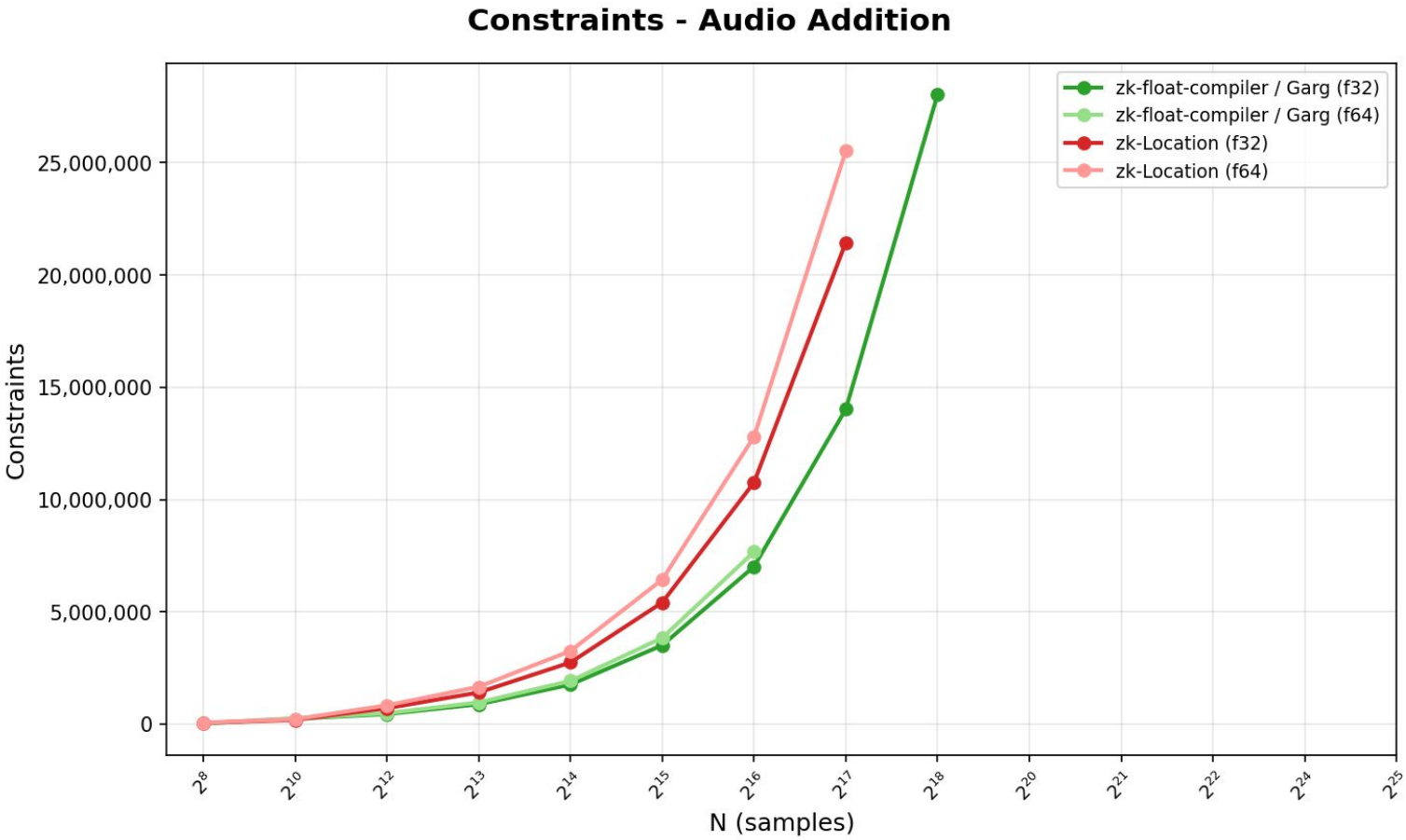
Results


Prover Time (s) - Audio Beep
zk-Location (f32)
Prover Time (s)
0
10
20
30
40
N (samples)
2^1
2^2
2^3
2^4
2^5
2^6
2^7
2^8
2^{10}
2^{12}

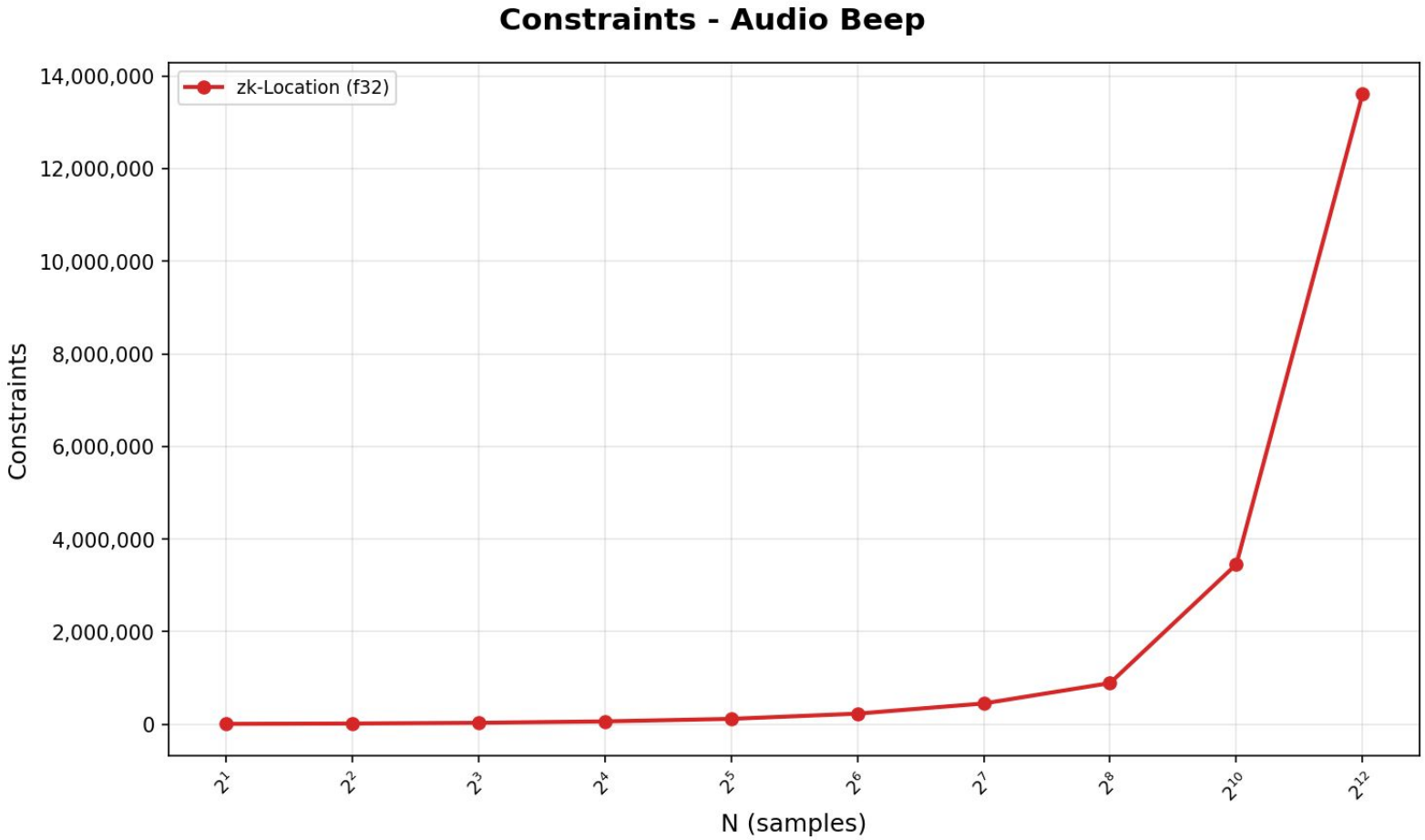
Results


Constraints - Audio Volume
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Constraints
30,000,000
25,000,000
20,000,000
15,000,000
10,000,000
5,000,000
0
2^{8}
2^{10}
2^{12}
2^{13}
2^{14}
2^{15}
2^{16}
2^{17}
2^{18}
2^{19}
2^{20}
2^{21}
2^{22}
2^{24}
2^{25}
N (samples)

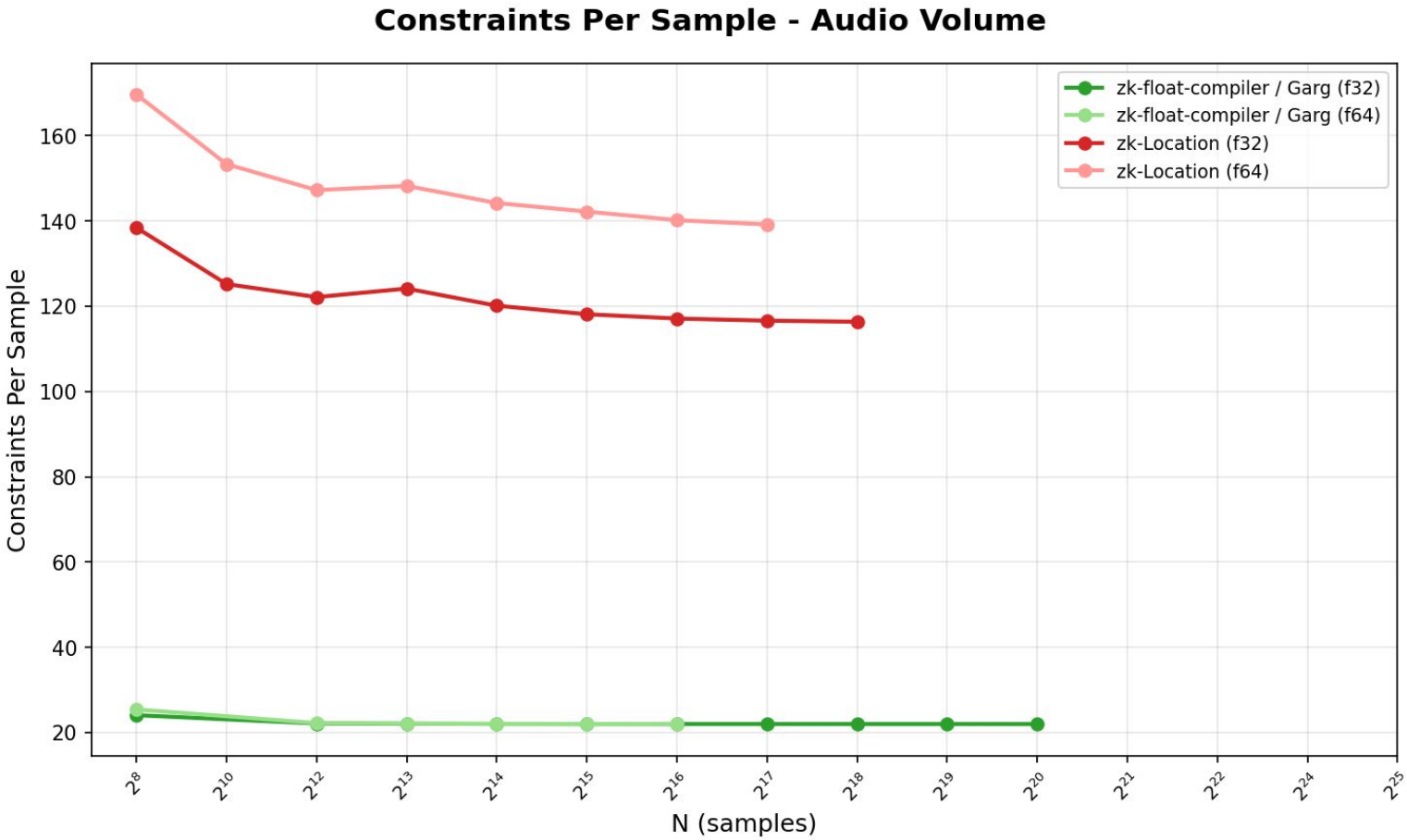
Results


Constraints - Audio Addition
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Constraints
25,000,000
20,000,000
15,000,000
10,000,000
5,000,000
0
N (samples)

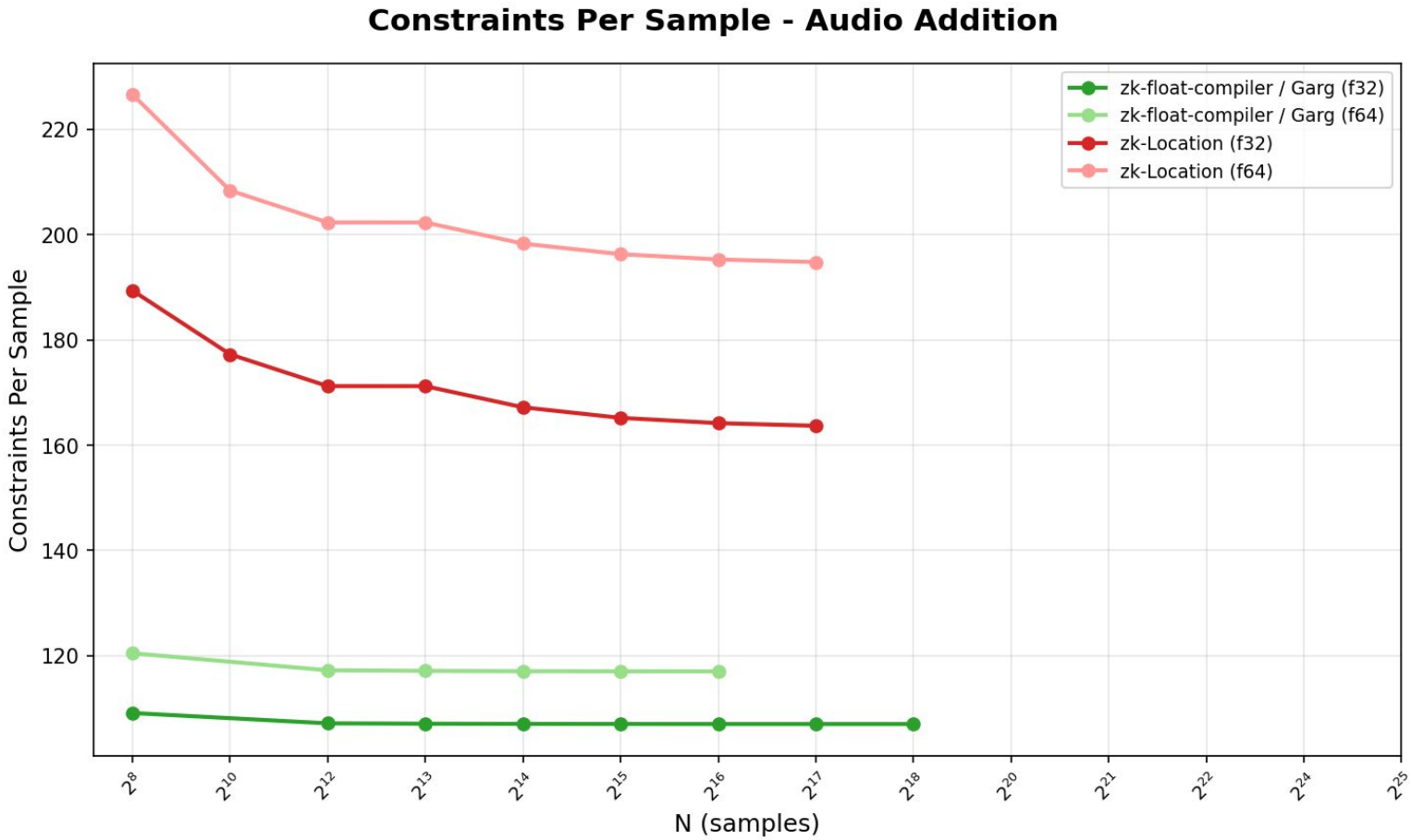
Results


Constraints - Audio Beep
zk-Location (f32)
14,000,000
12,000,000
10,000,000
8,000,000
6,000,000
4,000,000
2,000,000
0
Constraints
2^1
2^2
2^3
2^4
2^5
2^6
2^7
2^8
2^{10}
2^{12}
N (samples)

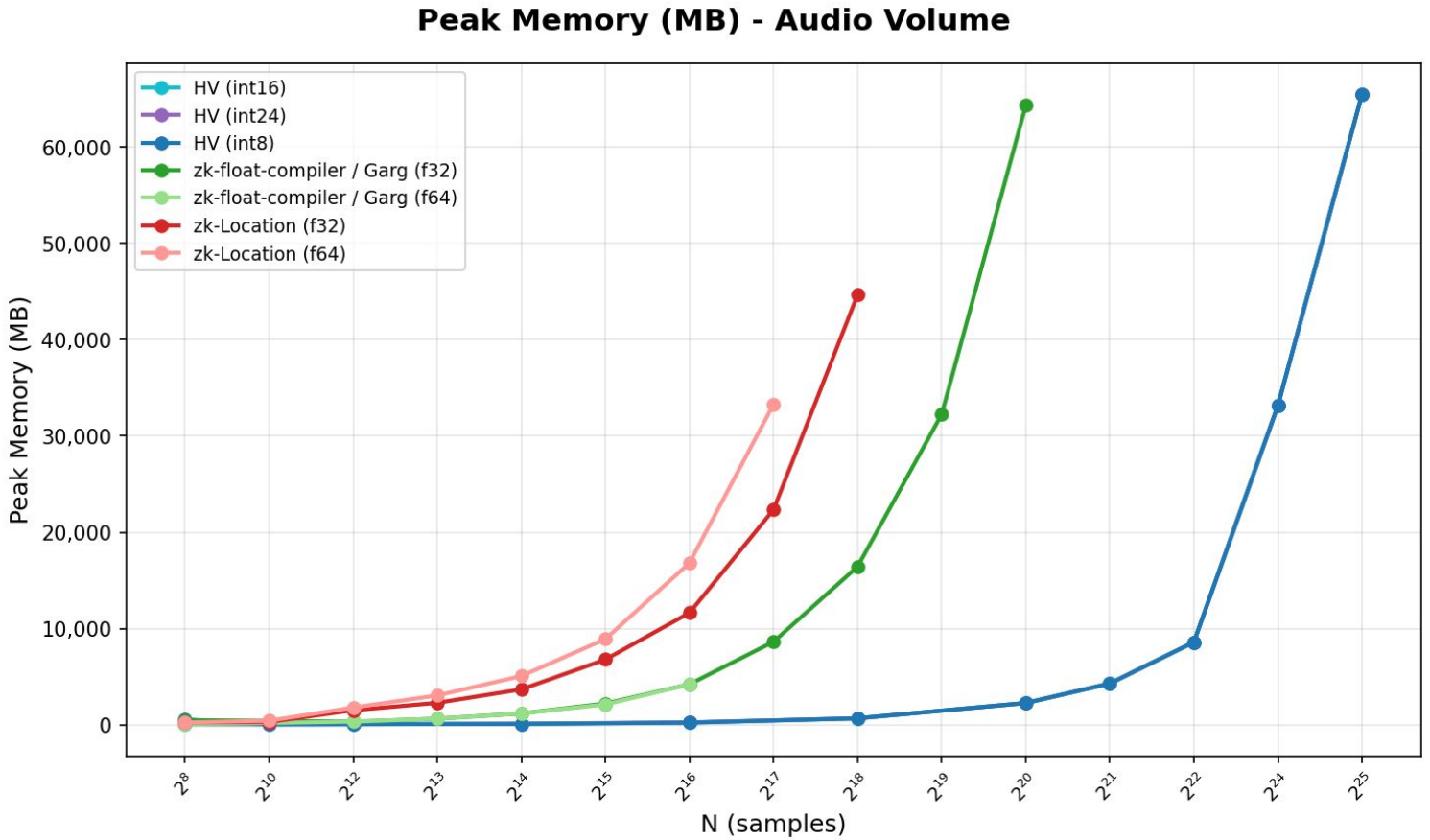
Results


Constraints Per Sample - Audio Volume
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Constraints Per Sample
160
140
120
100
80
60
40
20
2^{8}
2^{10}
2^{12}
2^{13}
2^{14}
2^{15}
2^{16}
2^{17}
2^{18}
2^{19}
2^{20}
2^{21}
2^{22}
2^{24}
2^{25}
N (samples)

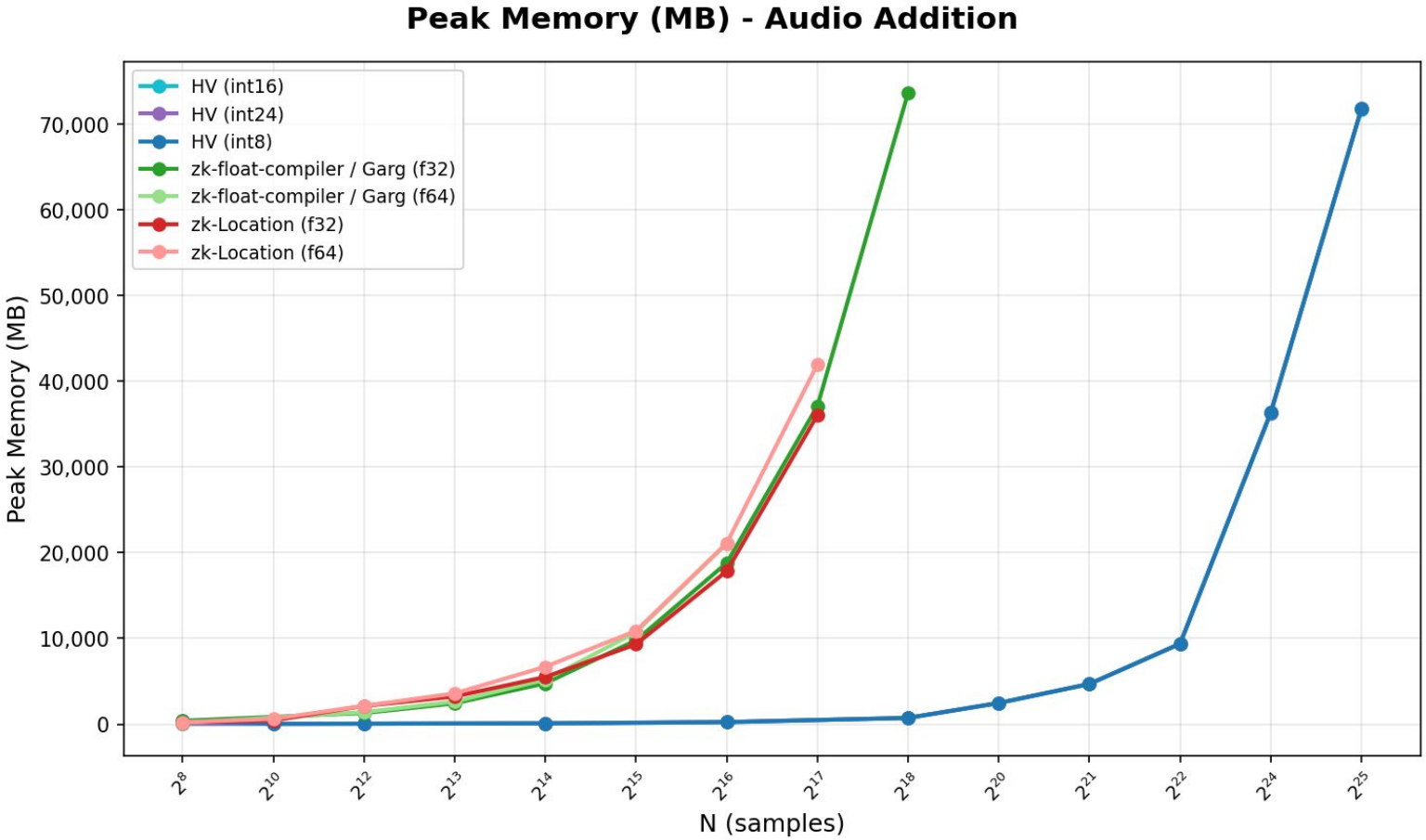
Results


Constraints Per Sample - Audio Addition
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Constraints Per Sample
220
200
180
160
140
120
N (samples)
2^8
2^10
2^12
2^13
2^14
2^15
2^16
2^17
2^18
2^20
2^21
2^22
2^24
2^25

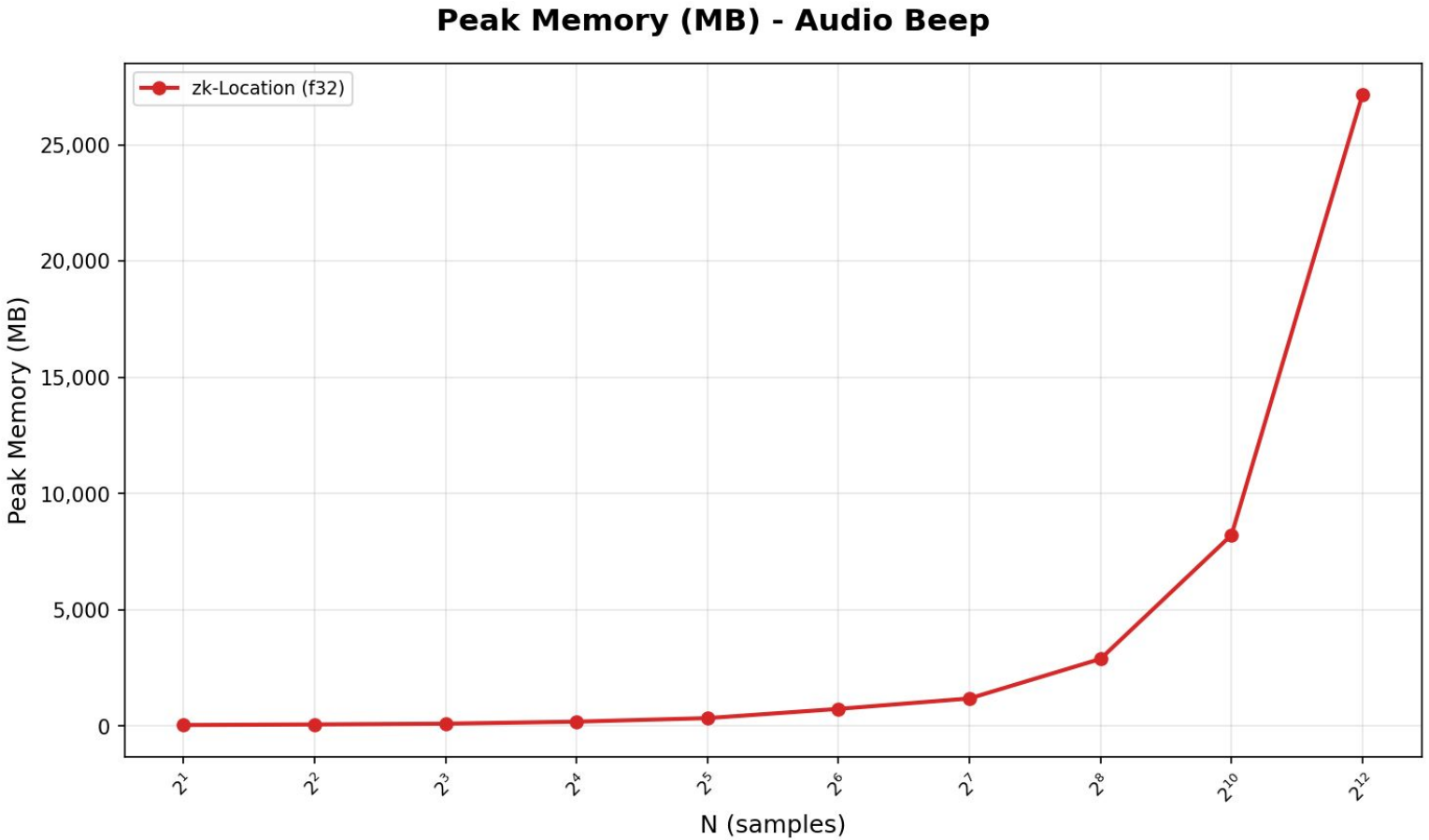
Results


Peak Memory (MB) - Audio Volume
HV (int16)
HV (int24)
HV (int8)
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Peak Memory (MB)
60,000
50,000
40,000
30,000
20,000
10,000
0
2^{8}
2^{10}
2^{12}
2^{13}
2^{14}
2^{15}
2^{16}
2^{17}
2^{18}
2^{19}
2^{20}
2^{21}
2^{22}
2^{24}
2^{25}
N (samples)

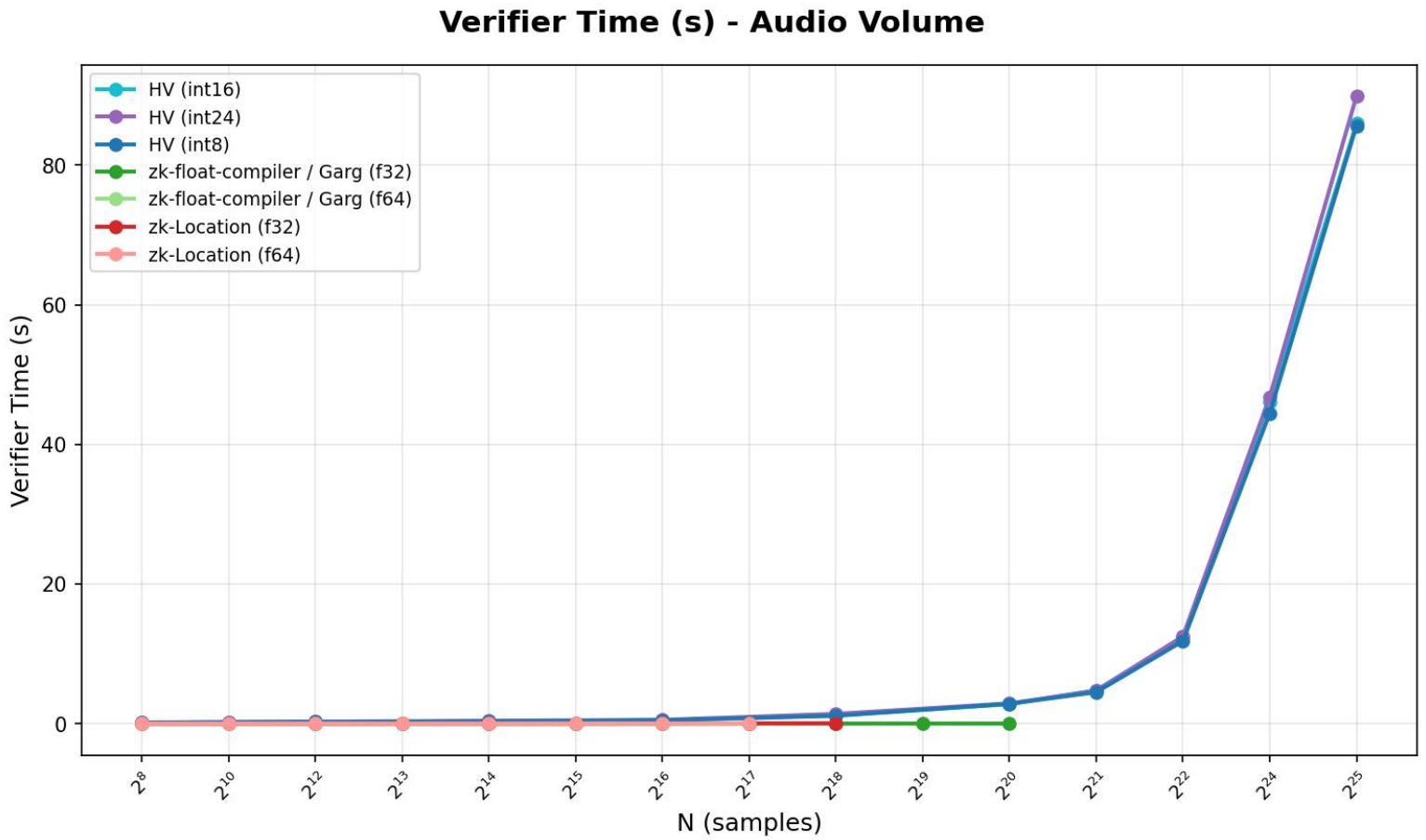
Results


Peak Memory (MB) - Audio Addition
HV (int16)
HV (int24)
HV (int8)
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Peak Memory (MB)
70,000
60,000
50,000
40,000
30,000
20,000
10,000
0
2^{8}
2^{10}
2^{12}
2^{13}
2^{14}
2^{15}
2^{16}
2^{17}
2^{18}
2^{20}
2^{21}
2^{22}
2^{24}
2^{25}
N (samples)

Results


Peak Memory (MB) - Audio Beep
zk-Location (f32)
Peak Memory (MB)
25,000
20,000
15,000
10,000
5,000
0
2^1
2^2
2^3
2^4
2^5
2^6
2^7
2^8
2^{10}
2^{12}
N (samples)

Results


Verifier Time (s) - Audio Volume
HV (int16)
HV (int24)
HV (int8)
zk-float-compiler / Garg (f32)
zk-float-compiler / Garg (f64)
zk-Location (f32)
zk-Location (f64)
Verifier Time (s)
0
20
40
60
80
N (samples)
2^{8}
2^{10}
2^{12}
2^{13}
2^{14}
2^{15}
2^{16}
2^{17}
2^{18}
2^{19}
2^{20}
2^{21}
2^{22}
2^{24}
2^{25}

Tradeoffs & Use Cases

- **HyperVerITAS** - 8/16/24 bit integer, fastest proving time and lowest memory usage. Higher proof size

- **zk-Location** - 64 bit and 32 bit full IEEE-754 w/ sin, cos, square root for complex transformations

- **Garg floating point compiler** - Lower constraint count for both addition and multiplication. Runs on higher sample sizes for low memory machines

ZK-Microphone

ZK-Microphone (Kang et al. 2023)

- Uses similar setting and threat model for audio provenance
- Limited implementation, could not benchmark
- Article states several minutes of proving time for ~30 seconds of audio with trim and concatenate transformations.

Outline

Problem

Background

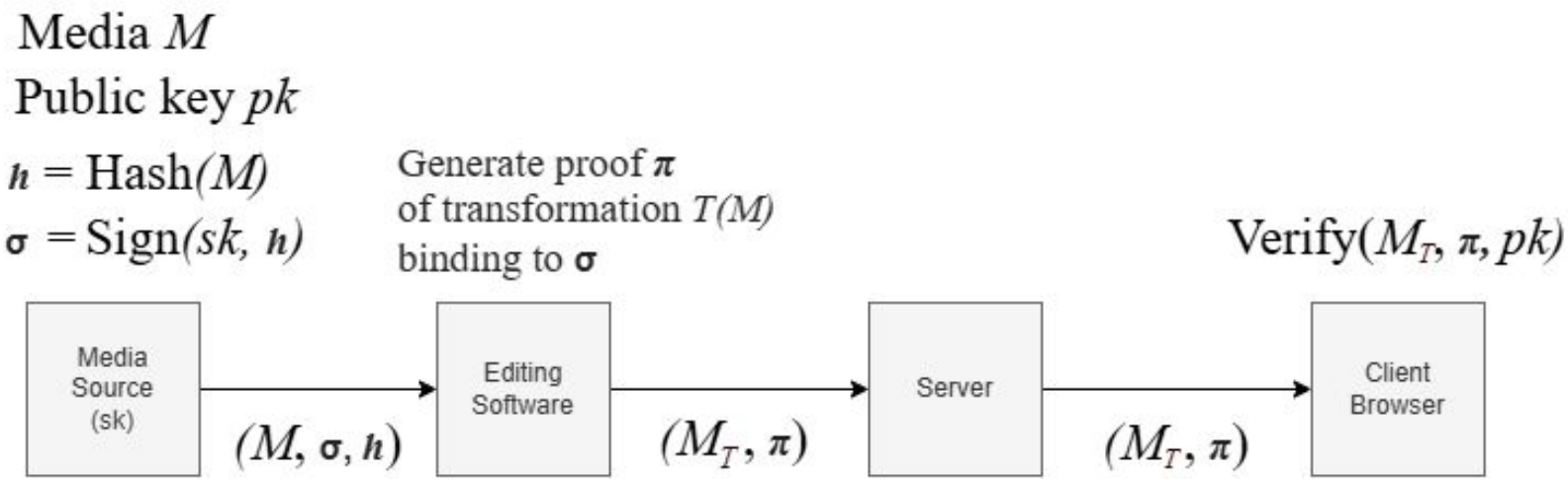
Cryptographic Tools

Baselines

Experiments & Results

End-to-end Demo

End-to-end Diagram



End-to-end Web Demo

Conclusion

Contributions:

- Garg et al. R1CS floating point compiler implementation
- Floating point ZKP comparison - Garg et al. vs zk-Location
- Extended HyperVerITAS to work with audio
- End-to-end image and audio provenance web prototype

Future Work:

- Linear argument for floating point ZKPs
- Trig functions to compute complex transformations

References

- Ernstberger, J., Zhang, C., Ciprian, L., Jovanovic, P., & Steinhorst, S. (2024, April 23). Zero-Knowledge location privacy via accurate Floating-Point SNARKs. arXiv.org. https://arxiv.org/abs/2404.14983
- Garg, S., Jain, A., Jin, Z., & Zhang, Y. (2022). Succinct zero knowledge for floating point computations. Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, 1203–1216. https://doi.org/10.1145/3548606.3560653
- Greiner, G., Mowery, T., & Soni, P. (2026, April 4). HyperVerITAS: Verifying image transformations at Scale on Boolean Hypercubes. IACR Cryptology ePrint Archive. https://eprint.iacr.org/2026/641
- Kang, D. (2023, June 23). Fighting AI-generated audio with attested microphones and zk-snarks: The attested audio experiment | by Daniel Kang | Medium. Medium. https://medium.com/@danieldkang/fighting-ai-generated-audio-with-attested-microphones-and-zk-snarks-the-attested-audio-experiment-d6ea0fc296ac

Questions?